SQL Assignment: LLM Agent Platform Database

Create a simple relational database for a platform where users build AI agents. The goal is to practice creating tables, primary keys, foreign keys, a many-to-many link table, basic constraints and simple SQL queries.

> **Background.** An *LLM agent* is a program that uses a large language model (LLM) to do tasks. Each agent belongs to a **user** and runs on one **model**. An agent is allowed to use some **tools** (web search, running code). Each task an agent does is a **run**, and during a run the agent makes **tool calls**. Models charge money per token (a token is a small piece of text).

1. Database Tables

users

Column	Meaning
`user_id`	Unique number of the user
`name`	User's full name
`email`	User's email

models

Column	Meaning
`model_id`	Unique number of the model
`model_name`	Name of the model
`price`	Price in USD per 1 million tokens

agents

Column	Meaning
`agent_id`	Unique number of the agent
`agent_name`	Name of the agent
`user_id`	The user who owns the agent
`model_id`	The model the agent uses

tools

Column	Meaning
`tool_id`	Unique number of the tool
`tool_name`	Name of the tool

agent_tools

Column	Meaning
`agent_id`	The agent
`tool_id`	A tool this agent is allowed to use

runs

Column	Meaning
`run_id`	Unique number of the run
`agent_id`	The agent that did the task
`task`	What the agent was asked to do
`tokens`	How many tokens the run used
`status`	Result of the run

tool_calls

Column	Meaning
`call_id`	Unique number of the tool call
`run_id`	The run during which the call happened
`tool_id`	The tool that was called
`status`	Result of the call

2. Relationships

- A user can own many agents. Each agent belongs to one user.
- A model can be used by many agents. Each agent uses one model.
- An agent can use many tools, and a tool can be used by many agents (many-to-many through `agent_tools`).
- An agent can have many runs. Each run belongs to one agent.
- A run can have many tool calls. Each tool call belongs to one run and is for one tool.
- The relationships must be implemented using foreign keys.

3. Required Constraints

- Every table must have a PRIMARY KEY. In `agent_tools` the primary key is the pair (agent_id, tool_id).
- All foreign keys must reference the appropriate tables.
- User email must be UNIQUE and NOT NULL. User name must be NOT NULL.
- Model name and tool name must be UNIQUE and NOT NULL.
- Agent name and run task must be NOT NULL.
- Model price and run tokens must be greater than or equal to 0.

- Run status must be `'COMPLETED'` or `'FAILED'`.
- Tool call status must be `'SUCCESS'` or `'ERROR'`.

4. Sample Data

- 2 users
- 2 models
- 2 agents
- 2 tools
- 3 agent–tool assignments
- 3 runs (at least one failed)
- 3 tool calls (at least one error)

5. Required SQL Queries

1. Display all users.
2. Display models that cost less than 10 USD per 1 million tokens.
3. Display all failed runs.
4. Display all tool calls that ended with an error.

6. What to Submit

- One `.sql` file that creates the tables, inserts the sample data and contains all the queries. It must run from top to bottom without errors.
- An ER diagram of the database (for example made in drawSQL or dbdiagram.io).